

Angular Pipes & Directives Exam Cheat Sheet

How to Identify Questions

If the question says format, display, or convert → use a Pipe. If it says style, highlight, dynamic behavior, or DOM manipulation → use a Directive.

Pipe Patterns

1. Text Formatting Pipe

```
transform(value: string): string {
  if (!value) return '';
  switch(value.toLowerCase()) {
    case 'tex_mex': return 'Tex-Mex';
    case 'bbq': return 'BBQ';
    default:
      return value.replace(/_/g, ' ')
        .split(' ')
        .map(w => w.charAt(0).toUpperCase() + w.slice(1))
        .join(' ');
  }
}
```

2. Range-Based Pipe (wait time, grades, price)

```
transform(value: number): string {
  if (value < 5) return 'Low';
  if (value <= 15) return 'Medium';
  return 'High';
}
```

3. Conditional Pipe (tenure, counts)

```
transform(value: number): string {
  if (value === 0) return 'None';
  if (value === 1) return '1 item';
  return `${value} items`;
}
```

Directive Patterns

1. Style Directive (based on input)

```
@Directive({ selector: '[appColor]' })
export class ColorDirective implements OnInit {
  @Input() appColor!: string;

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  ngOnInit() {
    const color = this.appColor === 'danger' ? 'red' : 'green';
    this.renderer.setStyle(this.el.nativeElement, 'background-color', color);
  }
}
```

```
}
```

2. Click Directive (change color on click)

```
@Directive({ selector: '[appClickColor]' })
export class ClickColorDirective {
  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('click')
  onClick() {
    this.renderer.setStyle(this.el.nativeElement, 'background-color', 'blue');
  }
}
```

3. Hover Directive (mouse enter/leave)

```
@Directive({ selector: '[appHover]' })
export class HoverDirective {
  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter')
  onEnter() {
    this.renderer.setStyle(this.el.nativeElement, 'background-color', 'yellow');
  }

  @HostListener('mouseleave')
  onLeave() {
    this.renderer.removeStyle(this.el.nativeElement, 'background-color');
  }
}
```

4. Status Badge Directive

```
@Directive({ selector: '[appStatus]' })
export class StatusDirective implements OnInit {
  @Input() appStatus!: boolean;

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  ngOnInit() {
    const text = this.appStatus ? 'OPEN' : 'CLOSED';
    const color = this.appStatus ? 'green' : 'gray';

    this.renderer.setProperty(this.el.nativeElement, 'innerText', text);
    this.renderer.setStyle(this.el.nativeElement, 'background-color', color);
  }
}
```

HTML Usage Examples

```
<!-- Pipe -->
<p>{{ truck.cuisine | cuisine }}</p>

<!-- Directive -->
<span [appColor]="truck.cuisine"></span>

<!-- Combined -->
<span [appColor]="truck.cuisine">
  {{ truck.cuisine | cuisine }}
</span>
```

Material Quick Templates

```
<table mat-table [dataSource]="dataSource" matSort></table>

<mat-card>
  <mat-card-title>Title</mat-card-title>
</mat-card>

<mat-form-field>
  <input matInput [(ngModel)]="value" name="value">
</mat-form-field>
```

Common Exam Mistakes

- Forgot to import PipeTransform - Forgot Renderer2 in directives - Missing @Input() - Forgot name="" in ngModel - Didn't apply pipe in HTML - Didn't use HostListener for events

Exam Strategy

1. Identify if it's a pipe or directive 2. Copy the correct pattern 3. Modify logic only 4. Apply it in HTML 5. Add imports if needed